Neural Discrete Representation Learning (VQ-VAE): A Reduced-Scale Re-implementation, Ablation and Code Prior on CIFAR-10

G. Sri Yasaswitha
Roll No. 24CS2015 (IDD)
Deep Learning 2026, Paper Implementation Project (Paper ID DL-081)

***Abstract*—I re-implement the vector-quantised variational autoencoder (VQ-VAE) of van den Oord et al. in PyTorch and study it on CIFAR-10 at small scale. The encoder turns each 32×32 image into an 8×8 grid of discrete codes, 64 integers in total. With a 512-entry codebook updated by exponential moving averages (EMA), the model reaches a test MSE of 0.01535 (24.2 dB PSNR) and uses all 512 codes (perplexity 407). Two ablations follow. Replacing the EMA update by gradient-based codebook learning collapses the codebook to 18 of 512 codes and raises the error 3.4×. Growing the codebook from 512 to 2048 entries lowers the error by under 1% while 70% of the codes stay unused. A PixelCNN prior trained over the codes reaches 4.63 nats per code (6.24 for a uniform guess) and produces plausible but blurry samples. All numbers come from one seed and a 10-epoch schedule.**

***Index Terms*—vector quantisation, VQ-VAE, discrete latent variables, codebook collapse, autoregressive prior, CIFAR-10.**

I. INTRODUCTION

An autoencoder squeezes an image through a bottleneck, and the form of that bottleneck decides what the latent code can later be used for. Continuous latents, as in the VAE [2], are easy to train, but they are usually paired with a simple Gaussian prior, and a strong decoder can learn to ignore the latent altogether (posterior collapse). van den Oord et al. [1] make the bottleneck discrete instead. Every encoder output vector is replaced by its nearest entry in a learned codebook, so an image becomes a grid of integers. The nearest-neighbour lookup has no gradient, so the gradient from the decoder is copied straight to the encoder [4]. A separate autoregressive model over the integers [3] then turns the autoencoder into a generator.

This report documents my re-implementation of that idea. The contributions are: (i) my own PyTorch implementation of the quantisation layer with the straight-through estimator, commitment loss and EMA codebook updates; (ii) a reduced-scale reproduction on CIFAR-10, with every change from the paper stated; (iii) two ablations, the codebook update rule and the codebook size, together with an analysis of codebook usage; and (iv) a PixelCNN prior over the codes, used to sample new images.

II. RELATED WORK

The VAE [2] learns continuous latents with a reparameterised Gaussian posterior. Discrete latents are harder to train because sampling a category is not differentiable. Relaxations such as the Gumbel-softmax [8] replace the sample by a smooth approximation, while VQ-VAE [1] avoids sampling: the assignment is deterministic (nearest codeword) and the gradient is passed through with the straight-through estimator [4]. The idea of copying gradients through a hard, non-differentiable unit goes back to [4]. Autoregressive models such as PixelCNN [3] are strong density estimators over discrete grids, which is why they are a natural prior for VQ-VAE codes. A later hierarchical version, VQ-VAE-2 [5], stacks several code grids at different scales; I do not implement it.

III. METHOD

A. Vector quantisation

The encoder maps an image x to a grid of vectors $z_e(x)$ of size H×W×D. A codebook holds K vectors $e_1, \ldots, e_K$ in R^D. At each grid position the encoder vector is replaced by its nearest codeword,

$$z_q = e_k, \quad k = \operatorname{argmin}_j \| z_e - e_j \|_2 \tag{1}$$

and the decoder reconstructs the image from the quantised grid z_q. Because argmin is not differentiable, the forward pass uses z_q while the backward pass treats the lookup as the identity:

$$z_q \leftarrow z_e + \mathrm{sg}[\, e_k - z_e \,] \tag{2}$$

where sg[·] is the stop-gradient operator. In the code this is the single line zq = z + (zq - z).detach().

B. Training objective

$$L = \| x - \hat{x} \|^2 + \| \mathrm{sg}[z_e] - e \|^2 + \beta \| z_e - \mathrm{sg}[e] \|^2 \tag{3}$$

The first term trains encoder and decoder to reconstruct the image. The second term (codebook loss) pulls the chosen codeword towards the encoder output and is the only term that updates the codebook. The third term (commitment loss) keeps the encoder output close to the codeword it was assigned, so the encoder does not drift away from the codebook. I use $\beta = 0.25$ as recommended in [1]. The squared error is a mean over pixels, divided by the pixel variance of the training set ($\sigma^2 = 0.2531$), so the loss has a standard scale.

C. EMA codebook update

The paper's appendix replaces the codebook loss by a moving average. For each code k, let n_k be the number of encoder vectors assigned to it in the batch and S_k the sum of those vectors. With decay $\gamma = 0.99$,

$$N_k \leftarrow \gamma N_k + (1-\gamma)n_k, \quad m_k \leftarrow \gamma m_k + (1-\gamma)S_k \quad (4)$$

$$e_k = m_k \,/\, N_k \quad (5)$$

where N_k is Laplace-smoothed so that no code has a zero count. The codebook is then not trained by gradients at all, and only the commitment term remains in Eq. (3).

D. Codebook usage and the prior

To see whether the codebook is used, I report the perplexity of the code distribution p_k (the fraction of positions assigned to code k):

$$\mathrm{PP} = \exp(\, -\, \Sigma_k\, p_k \ln p_k\,) \quad (6)$$

PP equals K when all codes are used equally and 1 when a single code is used. After training, each image is a grid of integers, which I model with a PixelCNN, $p(z) = \prod_i p(z_i \mid z_{<i})$, in raster order. Sampling draws the codes one position at a time, looks up their codewords, and runs the decoder.

IV. IMPLEMENTATION

Everything is in one PyTorch notebook. Seeds are fixed to 0 for Python, NumPy and PyTorch. I use CIFAR-10 (50,000 training and 10,000 test images) scaled to [−1, 1], with no augmentation. Standard blocks (Conv2d, Adam, torchvision) are used as is. The VectorQuantizer (nearest-code search, straight-through, both losses, EMA update, perplexity), the masked convolutions and the PixelCNN prior are my own code, following the description in [1].

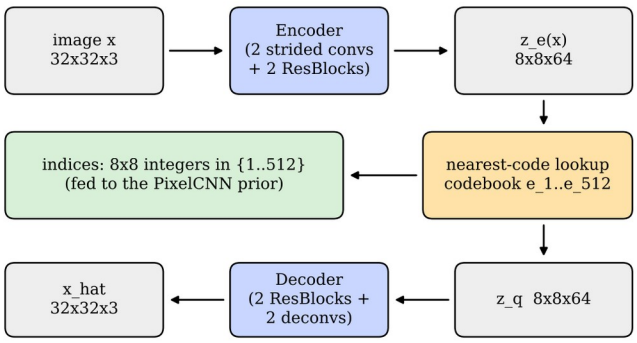


Fig. 1. Architecture. The quantiser turns the 8×8×64 encoder output into 8×8 integer indices; the decoder sees the matching codewords.

The encoder has two 4×4 stride-2 convolutions (128 channels), a 3×3 convolution, two residual blocks (3×3 convolution to 32 channels, 1×1 back to 128) and a 1×1 convolution to D = 64. The decoder mirrors it with two transposed convolutions. The whole VQ-VAE has about 0.96 M parameters (computed from the layer sizes). Squared distances to all K codewords are computed in the expanded form $|z|^2 - 2z\cdot e + |e|^2$. Codebooks start at U(−1/K, 1/K) and EMA uses γ = 0.99. Training uses Adam with learning rate 2×10^{-4}, batch size 128 and 10 epochs, with no schedule.

The prior embeds each code index, applies a 7×7 type-A masked convolution (which hides the current position), then eight residual 3×3 type-B masked convolutions (128 channels) and two 1×1 convolutions to 512 logits, about 2.1 M parameters. It is trained for 15 epochs with Adam (learning rate 10^{-3}) and cross-entropy, on the frozen codes of the K = 512 EMA model. Samples use temperature 1.

Changes from the paper. The paper uses larger networks trained far longer and reports results on more datasets (ImageNet, DeepMind Lab, speech) [1]. Here the network is small, training is 10 epochs, and only CIFAR-10 is used. The numbers below therefore cannot be compared directly with the paper's.

TABLE II
SETTINGS USED IN ALL EXPERIMENTS

Item	Value
Dataset	CIFAR-10, 50k train / 10k test
Latent grid	8×8, D = 64
Codebook size K	512 (ablation: 64, 2048)
β / EMA decay γ	0.25 / 0.99
VQ-VAE optimiser	Adam, lr 2×10^{-4}, batch 128
VQ-VAE epochs	10
Prior optimiser	Adam, lr 10^{-3}, batch 128, 15 epochs
Seed	0 (single run per setting)

V. EXPERIMENTS AND RESULTS

A. Metrics

I report test MSE per pixel on the [−1, 1] scale, and PSNR = $10 \log_{10}(4/\mathrm{MSE})$, which is derived from the MSE with a peak-to-peak range of 2. Perplexity and codes used (count > 0) come from code counts aggregated over the whole test set. The training loss in the logs is the normalised one. Its final value (0.0607 × 0.2531 = 0.01537) matches the test MSE (0.01535), so I see no generalisation gap for the autoencoder.

B. Reproduction: reconstructions

The main model (K = 512, EMA) reaches a test MSE of 0.01535 (24.2 dB) and uses all 512 codes with a perplexity of 407. An image is stored as 64 indices of 9 bits each, 576 bits against 24,576 bits for the raw 8-bit pixels, a 42.7× compression. Fig. 2 shows that colours and coarse shapes survive, while fine texture and small edges are smoothed.

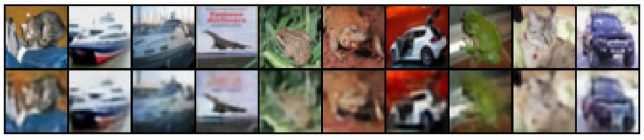


Fig. 2. Test images (top row) and their reconstructions from 64 integers (bottom row), K = 512 with EMA.

C. Ablation 1: EMA versus gradient codebook

Both runs use K = 512, the same seed and the same schedule. Only the codebook update differs. Table I and Fig. 3 show a clear failure of the gradient version: only 18 of 512 codes are ever selected on the test set, the perplexity is 8.4, and the MSE is 3.4× higher (18.9 dB against 24.2 dB). Its training loss levels off at 0.21, while the EMA model reaches 0.061.

TABLE I
ABLATION RESULTS ON CIFAR-10 TEST SET

Run	MSE	PSNR (dB)	Perplexity	Codes used
K=512, EMA	**0.01535**	**24.16**	**407.0**	**512 / 512**
K=512, gradient	0.05160	18.89	8.4	18 / 512
K=64, EMA	0.02143	22.71	57.0	64 / 64
K=2048, EMA	0.01521	24.20	449.8	602 / 2048

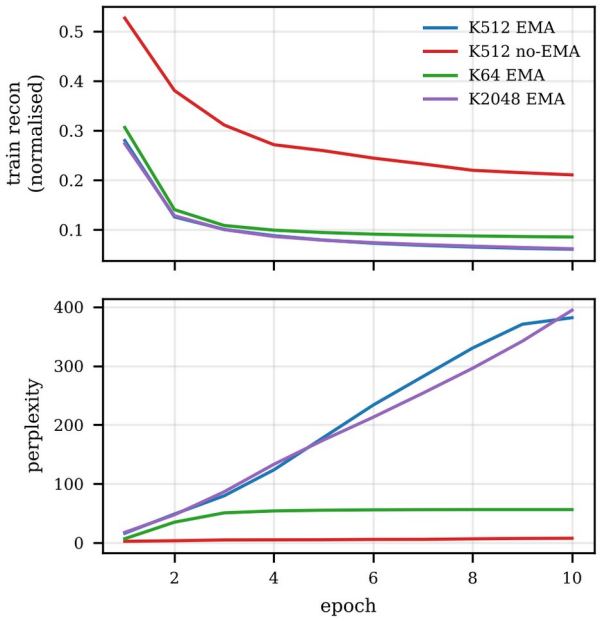


Fig. 3. Training curves (values from the training log). Top: normalised reconstruction loss. Bottom: codebook perplexity, averaged per epoch.

A plausible cause is that a codeword only moves when it is chosen. All codewords start near zero while encoder outputs do not, so a few codes win nearly every assignment and the others never receive a gradient. The EMA update normalises by the count of each code and avoids this in my runs. I did not test other fixes such as a higher codebook learning rate, k-means initialisation or restarting dead codes. The conclusion is therefore limited to my settings: it does not show that gradient-based codebooks cannot work.

D. Ablation 2: codebook size

With EMA, K = 64 is too small: every code is used, but the MSE is 40% higher than for K = 512 (22.7 dB). Going to K = 2048 gives almost nothing. The MSE improves by 0.9% (0.01521 against 0.01535), and only 602 of the 2048 codes (29%) are used. The test perplexity is higher for K = 2048 (449.8 against 407.0), but the extra spread does not buy a better reconstruction. The perplexity of K = 2048 was still rising at epoch 10 (Fig. 3), so a longer schedule could activate more codes. With one seed, a 0.9% difference may also be noise, which I did not measure. For this model and schedule K = 512 is enough.

E. Prior over codes and generation

The prior reaches a minimum test NLL of 4.632 nats per code at epoch 8 (Fig. 4). Two references help. A uniform guess costs ln 512 = 6.24 nats. The marginal code distribution of the test set costs ln 407 = 6.01 nats, from its perplexity. Conditioning on the earlier codes therefore saves about 1.4 nats per code. After epoch 8 the test NLL rises to 4.671 while the training NLL keeps falling to 4.384, which is overfitting.

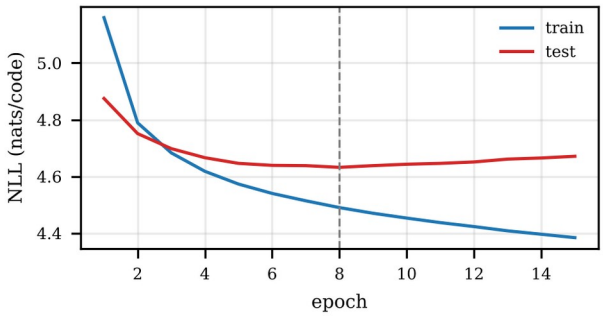


Fig. 4. PixelCNN prior: train and test NLL per code. The dashed line marks the best test epoch (8).

Fig. 5 shows 32 samples from the final (epoch-15) prior at temperature 1, decoded by the main VQ-VAE. They have plausible colour regions and layouts (sky, grass, ground), but few contain a recognisable object. I did not compute FID or another sample metric, so this judgement is qualitative only.

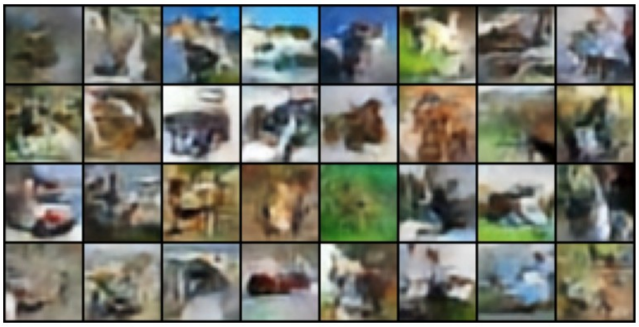


Fig. 5. Samples: codes drawn from the PixelCNN prior and decoded.

VI. FAILURE CASES AND WHAT DID NOT WORK

Codebook collapse. The gradient-based codebook used 18 of 512 codes (Section V-C). This was the clearest failure and the reason EMA is used for the main model.

Dead codes at large K. With K = 2048 about 70% of the codes were never selected, so the extra capacity was wasted.

Blur. Reconstructions lose fine texture, and samples are blurrier still. The small network, the 16× spatial reduction to an 8×8 grid and the short schedule all contribute, and I did not separate these effects.

Weak prior. The prior overfits after epoch 8, and I used the final model for sampling, not the best one. I added no early stopping, dropout or temperature tuning.

Limits of the study. One seed, 10 epochs, one dataset, no sweep over β, learning rate or network size, and no FID. The ablation differences are large enough to trust (EMA versus gradient), but the small ones (K = 512 versus 2048) are not.

VII. DISCUSSION

What the evidence supports. The EMA versus gradient gap (512 against 18 codes, 3.4× in MSE) is large, and it is hard to see a single seed alone producing it. The codebook-size result is weaker. Reconstruction error stops improving once K is large enough, and extra codes help only if they are used. Perplexity rose with K while the MSE stayed flat, which suggests that something other than the codebook limits quality here: the encoder and decoder capacity, the 8×8 resolution, or the short schedule. I did not test which, so this stays a hypothesis.

Relation to the paper. My results agree with the qualitative claims I could check: a 64-integer discrete code gives usable reconstructions, the EMA update of the appendix trains a working codebook, and an autoregressive prior over the codes gives samples. I do not claim to match the paper's numbers or image quality.

Threats to validity. There is no separate validation set. The "best" prior epoch in Fig. 4 was read from the test curve, so 4.632 is slightly optimistic. The samples, however, come from the final prior, so they are not affected by this choice. Hyperparameters were not tuned for any run, and the gradient variant in particular may be improved by tuning.

VIII. REPRODUCIBILITY

The Colab notebook runs top to bottom on a GPU runtime. Seeds are fixed (0), the constant EPOCHS sets the schedule (10 here), and the dataset is downloaded by torchvision. One cell trains the main model, one cell runs the three ablation models, and the last cells train the prior and write the figures. The repository README lists the setup, the commands and the results in Table I.

IX. CONCLUSION

I re-implemented VQ-VAE and tested it on CIFAR-10. A discrete 8×8 code with a 512-entry EMA codebook reconstructs images at 24.2 dB with 42.7× compression and uses every code. The ablations show that the EMA update matters a great deal in this setup (512 of 512 codes against 18 of 512) and that a larger codebook gave almost no gain at this scale. A PixelCNN prior over the codes beats a uniform guess by 1.6 nats per code and generates colour-and-layout-level samples. Next steps are code restarts for the gradient variant, longer training, a β sweep, several seeds, FID, and a hierarchical model as in VQ-VAE-2 [5].

REFERENCES

[1] A. van den Oord, O. Vinyals and K. Kavukcuoglu, "Neural discrete representation learning," in *Proc. NeurIPS*, 2017. arXiv:1711.00937.

[2] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. ICLR*, 2014. arXiv:1312.6114.

[3] A. van den Oord, N. Kalchbrenner, L. Espeholt, K. Kavukcuoglu, O. Vinyals and A. Graves, "Conditional image generation with PixelCNN decoders," in *Proc. NeurIPS*, 2016.

[4] Y. Bengio, N. Léonard and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," arXiv:1308.3432, 2013.

[5] A. Razavi, A. van den Oord and O. Vinyals, "Generating diverse high-fidelity images with VQ-VAE-2," in *Proc. NeurIPS*, 2019.

[6] A. Krizhevsky, "Learning multiple layers of features from tiny images," Tech. Rep., University of Toronto, 2009.

[7] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015. arXiv:1412.6980.

[8] E. Jang, S. Gu and B. Poole, "Categorical reparameterization with Gumbel-softmax," in *Proc. ICLR*, 2017. arXiv:1611.01144.